Notebook Ultra-Detaille - Gestionnaire de Budget : Finance Zen

(Version debutant - chaque ligne est expliquee)

C'est quoi ce projet exactement ?

C'est une **application web** qui te permet de gerer ton argent. Concretement :

- Tu peux **ajouter** des depenses (courses, loyer, metro...) et des revenus (salaire, freelance...)
- Tu vois ton **solde en temps reel** (combien il te reste = revenus - depenses)
- Tu as un **graphique en donut** qui te montre visuellement ou part ton argent
- Tu peux **filtrer** par categorie (alimentation, transport, loisirs...)
- Tu peux **switcher** entre mode clair et mode sombre

Pourquoi c'est un bon exercice ?

Parce qu'on touche a TOUT ce qu'un dev web junior doit savoir :

Competence	Ce qu'on apprend ici	Pourquoi c'est important
React	Composants, hooks, etat	C'est LE framework le plus demande en entreprise
TypeScript	Types, interfaces	Evite des bugs en production
Backend	API REST avec Express.js	Savoir creer un serveur qui traite des requetes
Communication	fetch, HTTP, JSON	Faire parler le front et le back ensemble
Persistance	Sauvegarder dans un fichier	Comprendre comment stocker des donnees
CSS moderne	Variables, themes, animations	Faire une interface qui a l'air pro
Graphiques	Chart.js, donut	Afficher des donnees visuellement

Les prerequis (ce que tu dois savoir avant)

1. **HTML/CSS de base** : ce qu'est une balise `<div>`, un selecteur CSS `.class`
2. **JavaScript de base** : variables (`let` , `const`), fonctions, tableaux, objets
3. **Node.js installe** : c'est le moteur qui fait tourner JavaScript en dehors du navigateur
4. **Terminal** : savoir ouvrir un terminal et taper des commandes

> Si tu connais pas encore React ou TypeScript, PAS DE PANIQUE ! Ce notebook explique chaque concept au fur et a mesure.

PARTIE 1 : Comment l'application est organisee (l'architecture)

Le modele "Client-Serveur" - c'est quoi ?

Imagine un restaurant :

- **Le client** (toi au resto) = ton **navigateur web** (Chrome, Firefox...)
- **Le serveur** (le cuistot en cuisine) = le **backend Express.js**
- **Le menu** que tu lis = le **frontend React** (l'interface visuelle)
- **La cuisine** ou on prepare = le **serveur** qui traite tes demandes
- **Le frigo** = le **fichier JSON** (la ou on stocke les donnees)

Pourquoi separer front et back ?

Imagine que tout soit dans le navigateur :

- Si tu fermes Chrome, toutes tes donnees disparaissent
- D'autres personnes ne pourraient pas acceder aux memes donnees

Avec un serveur separe :

- Les donnees sont **sauvegardees** meme si tu fermes le navigateur
- Plusieurs personnes pourraient utiliser l'app en meme temps

Le flux quand tu ajoutes une transaction

```
1. Tu remplis le formulaire dans le navigateur
2. Tu cliques "Ajouter"
3. React prepare les donnees en JSON
4. fetch() envoie une requete HTTP POST au serveur
5. Le serveur Express recoit la requete
6. Il ecrit dans le fichier transactions.json
7. Il repond "OK c'est fait" (status 201)
8. React met a jour l'affichage
```

Schema visuel

```
[TON NAVIGATEUR]                    [TON PC en arriere-plan]

React (interface)                   Express (serveur)
   port 5173          ---HTTP-->       port 3001
   "Affiche les                        "Recoit les
    boutons, le                         requetes et
```

```
    formulaire"                         lit/ecrit dans
                                        le fichier JSON"
                                         |
                                    data/transactions.json
                                    (les donnees sauvegardees)
```

C'est quoi un "port" ?

Un port c'est comme un **numero de porte dans un immeuble**. Ton PC a des milliers de ports.

- Le frontend ecoute sur la porte **5173** (c'est Vite qui choisit ce numero par defaut)
- Le backend ecoute sur la porte **3001** (c'est nous qui l'avons defini dans le code)

Ca permet aux deux programmes de tourner **en meme temps** sans se marcher dessus.

L'arborescence des fichiers (qui fait quoi)

```
gestionnaire-budget/
|
|-- index.html                  <-- La page HTML de base (le
squelette)
|-- package.json                <-- Liste des dependances +
commandes
|-- vite.config.ts              <-- Configuration de l'outil de
dev Vite
|-- server.js                   <-- BACKEND (le serveur Express)
|
|-- data/
|   |-- transactions.json       <-- Notre "base de donnees" (un
simple fichier)
|
|-- src/                        <-- FRONTEND (ce que
l'utilisateur voit)
|   |-- main.tsx                <-- Point d'entree : "demarre
React ici"
|   |-- App.tsx                 <-- Composant principal (le chef
d'orchestre)
|   |-- index.css               <-- Tous les styles visuels
|   |-- components/             <-- Les sous-composants React
|       |-- TransactionForm.tsx <-- Le formulaire d'ajout
|       |-- BudgetChart.tsx     <-- Le graphique en donut
|       |-- TransactionList.tsx <-- L'historique + filtres
```

Pourquoi decouper en composants ?

Au lieu de tout mettre dans un seul fichier App.tsx de 500+ lignes (impossible a lire), on decoupe en **3 briques** :

- `TransactionForm` : gere le formulaire tout seul
- `BudgetChart` : gere le graphique tout seul

- `TransactionList` : gere l'historique tout seul

Analogie : c'est comme construire une maison. Au lieu de tout faire d'un bloc, tu fais les murs, le toit, les fenetres separement. Si le toit a un probleme, tu touches que le toit.

PARTIE 2 : `package.json` - La carte d'identite du projet

C'est quoi ce fichier et pourquoi il est la ?

`package.json` c'est **le fichier le plus important** d'un projet Node.js. Il dit :

1. Comment s'appelle le projet
2. Quelles librairies il utilise
3. Quelles commandes on peut lancer

Analogie : c'est comme la liste d'ingredients d'une recette de cuisine. Sans elle, tu sais pas quoi acheter.

```json
{
  "name": "gestionnaire-budget",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "server": "node server.js",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "chart.js": "^4.4.7",
    "cors": "^2.8.6",
    "express": "^5.2.1",
    "react": "^19.2.0",
    "react-dom": "^19.2.0"
  }
}
```

Explication ligne par ligne

Ligne	Ca veut dire quoi	Pourquoi c'est la
`"name"`	Le nom du projet	Pour l'identifier
`"private": true`	Ce projet ne sera pas publie sur npm	C'est un projet perso, pas une librairie

Ligne	Ca veut dire quoi	Pourquoi c'est la
`"type": "module"`	Utiliser les imports modernes (`import/export`)	Au lieu de l'ancien `require()` de Node.js
`"dev": "vite"`	Quand tu tapes `npm run dev` , ca lance Vite	Vite sert le frontend en mode dev
`"server": "node server.js"`	`npm run server` lance le backend	Ca demarre Express.js
`"build": "tsc -b && vite build"`	`npm run build` compile pour la production	Pour deployer l'app en ligne

Les dependances (les librairies)

Librairie	A quoi elle sert	Pourquoi on en a besoin
`react`	Creer des interfaces dynamiques	C'est le coeur du frontend
`react-dom`	Connecte React au HTML de la page	React seul sait pas toucher au DOM
`express`	Creer un serveur HTTP en 5 lignes	Sans ca il faudrait 100+ lignes de code
`cors`	Autoriser les requetes entre ports differents	Sinon le navigateur bloque les requetes
`chart.js`	Dessiner des graphiques	Pour notre donut de repartition

Comment lancer le projet

```
# Etape 1 : installer les dependances (a faire UNE SEULE FOIS)
npm install
# -> ca telecharge toutes les librairies dans le dossier node_modules/

# Etape 2 : ouvrir DEUX terminaux separes

# Terminal 1 : lancer le backend
npm run server
# -> "Serveur demarre sur le port 3001"

# Terminal 2 : lancer le frontend
npm run dev
# -> "Local: http://localhost:5173"
```

> IMPORTANT : les deux terminaux doivent rester ouverts en meme temps !
> Le backend sert les donnees, le frontend les affiche.

PARTIE 3 : `index.html` et `vite.config.ts`

`index.html` - Le squelette de la page

```html
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Gestionnaire de Budget</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Ligne par ligne

Ligne	Explication	Pourquoi
`<!doctype html>`	Dit au navigateur "c'est du HTML5"	Standard obligatoire
`<html lang="fr">`	La langue de la page est le francais	Aide les lecteurs d'ecran et le SEO
`<meta charset="UTF-8">`	Encodage des caracteres	Pour afficher les accents (e, a, etc.)
`<meta name="viewport">`	La page s'adapte a l'ecran	INDISPENSABLE pour le responsive mobile
`<div id="root">`	**Le conteneur ou React va s'injecter**	React remplace le contenu de cette div
`<script src="/src/main.tsx">`	Charge le fichier TypeScript principal	C'est le point d'entree de tout le JS

> **Le truc a retenir** : React ne genere pas le HTML. Il REMPLACE le contenu du `<div id="root">`.

`vite.config.ts` - La configuration de Vite

```typescript
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
    plugins: [react()],
    server: {
```

```
        proxy: {
            '/api': 'http://localhost:3001'
        }
    }
})
```

C'est quoi Vite ?

Vite c'est un **outil de developpement** pour le frontend qui fait 3 choses :

1. **Compile** le TypeScript en JavaScript (le navigateur comprend pas TypeScript)
2. **Recharge** la page automatiquement quand tu modifies un fichier (hot reload)
3. **Sert** les fichiers sur le port 5173

C'est quoi le proxy et pourquoi c'est important ?

Le navigateur a une **securite** appele CORS qui **bloque** les requetes entre 2 ports differents (5173 -> 3001). Le proxy contourne ca :

```
SANS proxy :
  fetch('http://localhost:3001/api/transactions')
  --> Le navigateur dit "STOP ! Ports differents = BLOQUE !"

AVEC proxy :
  fetch('/api/transactions')
  --> Vite intercepte, voit que ca commence par '/api'
  --> Il redirige vers http://localhost:3001
  --> Le navigateur voit rien, ca passe !
```

Analogie : c'est comme un traducteur. Au lieu de parler directement a quelqu'un qui parle une autre langue, tu passes par le traducteur.

PARTIE 4 : `data/transactions.json` - La base de donnees

Pourquoi un fichier JSON ?

Dans un vrai projet pro, on utilise une base de donnees (PostgreSQL, MongoDB...). Mais pour un projet d'apprentissage, un fichier JSON c'est parfait :

- Pas besoin d'installer quoi que ce soit
- On peut l'ouvrir et le lire facilement
- Ca marche exactement pareil qu'une BDD pour notre cas

La structure d'une transaction

```json
[
    {
        "id": "1772198316422yveub4k0zl",
        "name": "Courses supermarche",
        "amount": 85.50,
        "type": "depense",
        "category": "alimentation",
        "date": "2026-02-03"
    }
]
```

Chaque champ explique

Champ	Type	Exemple	A quoi ca sert	Pourquoi ce format
`id`	string	`"1772198..."`	Identifier de facon unique chaque transaction	Genere avec `Date.now() + Math.random()` pour etre unique
`name`	string	`"Courses"`	Description lisible par l'humain	C'est ce qu'on affiche dans la liste
`amount`	number	`85.50`	Le montant en euros	Toujours positif, le `type` dit si c'est + ou -
`type`	string	`"depense"`	Revenu ou depense	Permet de calculer le solde (revenus - depenses)
`category`	string	`"alimentation"`	La categorie de la transaction	Pour filtrer et pour le graphique en donut
`date`	string	`"2026-02-03"`	La date de la transaction	Format ISO : AAAA-MM-JJ (annee-mois-jour)

> **Pourquoi `amount` est toujours positif ?** Parce que c'est le champ `type` qui determine le signe. Une depense de 50 euros, c'est `amount: 50, type: "depense"`. Le `-` est ajoute a l'affichage.

PARTIE 5 : `server.js` - Le Backend (le serveur)

C'est le fichier qui fait tourner le serveur. Il attend des requetes HTTP et y repond.

5.1 - Les imports et la configuration initiale

```
// on importe les librairies dont on a besoin
import express from 'express'       // pour creer le serveur facilement
import cors from 'cors'             // pour autoriser les requetes
cross-origin
import fs from 'fs'                 // pour lire/ecrire des fichiers
```

```
(File System)
import path from 'path'             // pour manipuler les chemins de
fichiers
import { fileURLToPath } from 'url' // pour reconstruire __dirname en
ESM

const app = express()    // on cree l'application Express
const PORT = process.env.PORT || 3001  // le port, 3001 par defaut

// en mode ESM (type: "module"), __dirname n'existe pas
// donc on le reconstruit manuellement
const __filename = fileURLToPath(import.meta.url)
const __dirname = path.dirname(__filename)

// le chemin vers notre fichier de donnees
const DATA_DIR = path.join(__dirname, 'data')
const DATA_FILE = path.join(DATA_DIR, 'transactions.json')

// on verifie que le dossier data/ existe, sinon on le cree
if (!fs.existsSync(DATA_DIR)) {
    fs.mkdirSync(DATA_DIR, { recursive: true })
}

// middlewares : des fonctions qui s'executent AVANT chaque requete
app.use(cors())           // autorise les requetes de n'importe quelle
origine
app.use(express.json())   // parse automatiquement le JSON dans le body
des requetes
```

Pourquoi `cors()` ?

CORS = Cross-Origin Resource Sharing. Le navigateur bloque par securite les requetes entre domaines/ports differents. `cors()` dit au serveur "autorise tout le monde a me parler".

Pourquoi `express.json()` ?

Quand le frontend envoie des donnees (ex: une nouvelle transaction), elles arrivent sous forme de texte JSON. `express.json()` transforme automatiquement ce texte en objet JavaScript utilisable.

5.2 - Les fonctions utilitaires (lire/ecrire le fichier)

```
// lire toutes les transactions depuis le fichier
function readTransactions() {
    try {
        if (!fs.existsSync(DATA_FILE)) return []   // si le fichier
existe pas -> tableau vide
        const data = fs.readFileSync(DATA_FILE, 'utf-8')  // lire le
contenu comme du texte
        return JSON.parse(data)   // transformer le texte JSON en
tableau JavaScript
```

```
    } catch (err) {
        return []    // si erreur (fichier corrompu etc) -> tableau
vide
    }
}

// ecrire les transactions dans le fichier
function writeTransactions(transactions) {
    // JSON.stringify transforme un objet JS en texte JSON
    // null = pas de fonction de remplacement
    // 4 = indentation de 4 espaces (pour la lisibilite)
    fs.writeFileSync(DATA_FILE, JSON.stringify(transactions, null, 4),
'utf-8')
}
```

C'est quoi `try/catch` ?

C'est un filet de securite. Si le code dans `try` plante (fichier introuvable, JSON invalide...), au lieu de crasher le serveur, on execute le code dans `catch`. Ici on retourne juste un tableau vide.

5.3 - Les routes de l'API REST

C'est quoi une API REST ?

REST c'est une convention pour organiser les URLs d'un serveur. On utilise les **verbes HTTP** (GET, POST, DELETE) pour dire ce qu'on veut faire :

Verbe HTTP	Signification	Exemple
GET	"Donne-moi des donnees"	Recuperer la liste des transactions
POST	"Ajoute cette donnee"	Creer une nouvelle transaction
DELETE	"Supprime cette donnee"	Effacer une transaction

Les 4 routes de notre API

```
// ROUTE 1 : GET /api/transactions -> recuperer TOUTES les transactions
app.get('/api/transactions', (req, res) => {
    const transactions = readTransactions()
    res.json(transactions)    // on renvoie le tableau en JSON
})
// req = la requete du client (ce qu'il demande)
// res = la reponse du serveur (ce qu'on lui renvoie)

// ROUTE 2 : POST /api/transactions -> AJOUTER une transaction
app.post('/api/transactions', (req, res) => {
    const transactions = readTransactions()
    const newTransaction = req.body    // les donnees envoyees par le
frontend
```

```
    // on genere un id si y'en a pas (securite)
    if (!newTransaction.id) {
        newTransaction.id = Date.now().toString() +
Math.random().toString(36).substring(2)
    }
    // on ajoute la date du jour si elle est pas renseignee
    if (!newTransaction.date) {
        newTransaction.date = new Date().toISOString().split('T')[0]
    }

    transactions.unshift(newTransaction)  // unshift = ajouter AU DEBUT
du tableau
    writeTransactions(transactions)
    res.status(201).json(newTransaction)  // 201 = "Created" (creation
reussie)
})

// ROUTE 3 : DELETE /api/transactions/:id -> SUPPRIMER une transaction
par son id
app.delete('/api/transactions/:id', (req, res) => {
    const transactions = readTransactions()
    // filter garde seulement les transactions dont l'id est DIFFERENT
de celui qu'on veut supprimer
    const filtered = transactions.filter(t => t.id !== req.params.id)

    if (filtered.length === transactions.length) {
        // si la longueur a pas change, c'est que l'id existait pas
        return res.status(404).json({ error: 'Transaction pas trouvee'
})
    }

    writeTransactions(filtered)
    res.json({ message: 'Transaction supprimee' })
})

// ROUTE 4 : DELETE /api/transactions -> TOUT EFFACER
app.delete('/api/transactions', (req, res) => {
    writeTransactions([])     // on ecrit un tableau vide
    res.json({ message: 'Toutes les transactions supprimees' })
})

// on demarre le serveur
app.listen(PORT, () => {
    console.log(`Serveur demarre sur le port ${PORT}`)
})
```

C'est quoi `:id` dans l'URL ?

C'est un **parametre dynamique**. Quand le frontend appelle `DELETE /api/transactions/abc123`, Express met `"abc123"` dans `req.params.id`. C'est comme un trou a remplir dans l'URL.

PARTIE 6 : `src/main.tsx` - Le point de demarrage de React

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
    <StrictMode>
        <App />
    </StrictMode>,
)
```

Ligne par ligne

Ligne	Explication
`import { StrictMode }`	Un outil de React qui detecte les erreurs en dev
`import { createRoot }`	La fonction qui "branche" React sur le HTML
`import './index.css'`	Charge tous les styles CSS
`import App`	Charge notre composant principal
`document.getElementById('root')`	Trouve le `<div id="root">` dans index.html
`!`	Dit a TypeScript "je suis SUR que cet element existe"
`<StrictMode>`	Active les verifications en mode dev
`<App />`	Affiche notre composant App dans la page

Pourquoi `StrictMode` ?

En mode dev, il execute certaines fonctions 2 fois pour detecter des bugs. Ca ne se produit PAS en production. Si tu vois des logs en double dans la console, c'est normal !

PARTIE 7 : `src/App.tsx` - Le composant principal

C'est le **chef d'orchestre** de l'application. Il gere :

- L'etat global (la liste des transactions)
- Les appels API vers le backend
- L'assemblage des 3 sous-composants

7.1 - Les imports

```
import { useReducer, useEffect, useState, useCallback } from 'react'
import TransactionForm from './components/TransactionForm'
import BudgetChart from './components/BudgetChart'
import TransactionList from './components/TransactionList'
```

C'est quoi les "hooks" React ?

Les hooks sont des fonctions speciales de React qui commencent par `use`. Ils permettent de donner des "super-pouvoirs" a un composant :

Hook	Analogie	A quoi ca sert
`useState`	Un post-it	Memorise UNE valeur qui peut changer (ex: le theme)
`useReducer`	Un cahier de comptes	Comme useState mais pour des etats complexes avec plusieurs actions
`useEffect`	Un robot qui surveille	Execute du code quand quelque chose change
`useCallback`	Un raccourci clavier	Memorise une fonction pour eviter de la recreer a chaque rendu

7.2 - Les types TypeScript et les constantes

```
// les categories disponibles pour classer les transactions
export const CATEGORIES = [
    'alimentation', 'transport', 'loisirs', 'logement',
    'sante', 'salaire', 'freelance', 'autre',
] as const
```

Pourquoi `as const` ?

Sans `as const`, TypeScript pense que c'est un tableau de `string` quelconque (on pourrait mettre "pizza" dedans). Avec `as const`, TypeScript sait que c'est EXACTEMENT ces 8 valeurs et rien d'autre. Ca empeche les erreurs.

```
// ce type ne peut etre QUE l'une des 8 categories ci-dessus
export type Category = typeof CATEGORIES[number]

// ce type ne peut etre QUE 'revenu' ou 'depense'
export type TransactionType = 'revenu' | 'depense'

// la forme d'une transaction (comme un modele/template)
export type Transaction = {
    id: string           // identifiant unique
    name: string         // description ("Courses", "Salaire"...)
    amount: number       // montant en euros
    type: TransactionType  // 'revenu' ou 'depense'
    category: Category     // une des 8 categories
    date: string           // date au format "2026-03-08"
}
```

Pourquoi TypeScript et les types ?

Analogie : les types c'est comme un plan de maison. AVANT de construire, tu sais que la cuisine a une porte, 2 fenetres, etc. Si quelqu'un essaie de mettre 3 portes, le plan dit "ERREUR !".

Sans TypeScript, tu pourrais ecrire `transaction.amont` (faute de frappe) et tu ne le verras qu'en production. Avec TypeScript, l'erreur est detectee **pendant que tu codes**.

`export` c'est quoi ?

`export` permet aux autres fichiers d'importer cette valeur. Sans `export`, le type `Transaction` ne serait utilisable QUE dans `App.tsx`. Avec, les composants enfants peuvent l'utiliser aussi.

7.3 - Le Reducer : gerer l'etat de l'application

C'est quoi un "state" (etat) ?

L'etat c'est **toutes les donnees qui peuvent changer** dans l'app. Ici c'est la liste des transactions. Quand l'etat change, React **re-affiche** automatiquement la page.

C'est quoi un reducer et pourquoi pas juste useState ?

`useState` c'est bien pour UNE valeur simple (ex: le theme "light"/"dark"). Mais pour la liste des transactions, on a **4 actions differentes** : ajouter, supprimer, charger tout, tout effacer.

Le reducer c'est comme un **distributeur automatique** :

- Tu mets une piece (dispatch une action)
- Tu choisis un bouton (le type d'action)
- La machine te donne le resultat (le nouvel etat)

```
// les 4 actions possibles que le reducer peut recevoir
type BudgetAction =
    | { type: 'ADD_TRANSACTION';   payload: Transaction }    // ajouter
    | { type: 'DELETE_TRANSACTION'; payload: string }         //
supprimer (payload = l'id)
    | { type: 'LOAD_TRANSACTIONS'; payload: Transaction[] }   //
charger depuis le serveur
    | { type: 'RESET_TRANSACTIONS' }                          // tout
vider

// le reducer : il recoit l'etat actuel + une action, et retourne le
NOUVEL etat
```

```
function budgetReducer(state: BudgetState, action: BudgetAction):
BudgetState {
    switch (action.type) {
        case 'ADD_TRANSACTION': {
            // on met la nouvelle transaction EN PREMIER
(...state.transactions = le reste)
            const updated = [action.payload, ...state.transactions]
            return { transactions: updated }
        }
        case 'DELETE_TRANSACTION': {
            // filter garde toutes les transactions SAUF celle avec
l'id a supprimer
            const updated = state.transactions.filter(t => t.id !==
action.payload)
            return { transactions: updated }
        }
        case 'LOAD_TRANSACTIONS':
            // on remplace tout par ce que le serveur nous a renvoye
            return { transactions: action.payload }
        case 'RESET_TRANSACTIONS':
            // on met un tableau vide
            return { transactions: [] }
        default:
            return state  // si action inconnue, on change rien
    }
}
```

C'est quoi `...` (le spread operator) ?

```
const ancien = ['a', 'b', 'c']
const nouveau = ['z', ...ancien]
// resultat : ['z', 'a', 'b', 'c']
```

Le `...` "etale" le contenu du tableau. Ca permet de creer un NOUVEAU tableau sans modifier l'ancien.

Pourquoi on cree un NOUVEAU tableau a chaque fois ?

React compare l'ancien et le nouveau state. Si c'est le MEME objet en memoire (meme reference), React pense que rien n'a change et ne re-affiche rien. En creant un nouveau tableau, React detecte le changement.

7.4 - Le composant App : theme, chargement, et handlers

```
function App() {
    // --- GESTION DU THEME ---
    // useState avec une fonction d'initialisation (lazy init)
    const [theme, setTheme] = useState<'light' | 'dark'>(() => {
        // on regarde si un theme est deja sauvegarde dans le
navigateur
```

```
        const saved = localStorage.getItem(THEME_KEY)
        return (saved === 'dark') ? 'dark' : 'light'
    })

    // --- LE REDUCER (l'etat principal) ---
    // state = { transactions: [...] }
    // dispatch = la fonction pour envoyer des actions au reducer
    const [state, dispatch] = useReducer(budgetReducer, { transactions:
[] })
```

C'est quoi `localStorage` ?

C'est un **petit espace de stockage** dans ton navigateur. Les donnees y restent meme si tu fermes l'onglet. On l'utilise ici pour memoriser si l'utilisateur prefere le theme sombre ou clair.

```
    // --- EFFETS (useEffect) ---

    // Effet 1 : quand le theme change, on l'applique sur la page
    useEffect(() => {
        document.documentElement.setAttribute('data-theme', theme)
        localStorage.setItem(THEME_KEY, theme)
    }, [theme])    // [theme] = "execute ca SEULEMENT quand `theme`
change"

    // Effet 2 : au demarrage, charger les transactions depuis le
serveur
    useEffect(() => {
        loadTransactions().then(loaded => {
            dispatch({ type: 'LOAD_TRANSACTIONS', payload: loaded })
        })
    }, [])    // [] = "execute ca UNE SEULE FOIS au demarrage"
```

Le tableau de dependances `[...]`

- `[theme]` = "re-execute quand `theme` change"
- `[]` = "execute une seule fois au montage du composant"
- Pas de tableau = "execute a chaque rendu" (dangereux !)

```
    // --- CALCULS DERIVES ---
    const totalRevenu = state.transactions
        .filter(t => t.type === 'revenu')       // garder que les
revenus
        .reduce((sum, t) => sum + t.amount, 0)  // faire la somme

    const totalDepense = state.transactions
        .filter(t => t.type === 'depense')
        .reduce((sum, t) => sum + t.amount, 0)

    const solde = totalRevenu - totalDepense     // le solde final
```

C'est quoi `.filter()` et `.reduce()` ?

- `filter()` : garde seulement les elements qui passent le test. Ex: garder que les revenus.
- `reduce()` : "reduit" un tableau a une seule valeur. Ici, on additionne tous les montants.

```
// Exemple concret :
[10, 20, 30].reduce((somme, valeur) => somme + valeur, 0)
// Etape 1 : somme=0, valeur=10 -> 0+10 = 10
// Etape 2 : somme=10, valeur=20 -> 10+20 = 30
// Etape 3 : somme=30, valeur=30 -> 30+30 = 60
// Resultat : 60
```

7.5 - Les handlers (fonctions appelees quand l'utilisateur agit)

```
    // Quand l'utilisateur ajoute une transaction
    async function handleAddTransaction(name, amount, type, category) {
        // 1. On cree l'objet transaction
        const newTransaction = {
            id: Date.now().toString() +
Math.random().toString(36).substring(2),
            name, amount, type, category,
            date: new Date().toISOString().split('T')[0],  // "2026-03-
08"
        }

        try {
            // 2. On envoie au serveur via POST
            await fetch(API_URL, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(newTransaction),
            })
            // 3. Si OK, on met a jour le state React
            dispatch({ type: 'ADD_TRANSACTION', payload: newTransaction
})
        } catch {
            console.error('Erreur...')
        }
    }
```

C'est quoi `async/await` ?

Les appels reseau (fetch) prennent du temps. `await` dit "attends que ca soit fini avant de continuer". Sans `await`, le code continuerait SANS attendre la reponse du serveur.

C'est quoi `fetch()` ?

`fetch` c'est la fonction native du navigateur pour envoyer des requetes HTTP. C'est comme envoyer un courrier au serveur.

Pourquoi `headers: { 'Content-Type': 'application/json' }` ?

Ca dit au serveur "attention, ce que je t'envoie c'est du JSON". Sans ca, le serveur ne saurait pas comment lire les donnees.

7.6 - Le JSX (le HTML de React)

```
    return (
        <div id="app">
            {/* Le header avec le logo et le bouton theme */}
            <header className="app-header">
                <div className="logo-section">
                    <img src="/fiance_zen.png" alt="Finance Zen" />
                    <h1>Finance Zen</h1>
                </div>
                <button onClick={toggleTheme}>
                    {theme === 'light' ? 'Sombre' : 'Clair'}
                </button>
            </header>

            {/* La carte du solde */}
            <div className="balance-card">
                <div className="label">Solde actuel</div>
                <div className="amount">{formatMoney(solde)}</div>
            </div>

            {/* Les deux boxes revenus/depenses */}
            <div className="summary-row">
                <div>Revenus: +{formatMoney(totalRevenu)}</div>
                <div>Depenses: -{formatMoney(totalDepense)}</div>
            </div>

            {/* La grille formulaire + graphique */}
            <div className="main-grid">
                <TransactionForm onSubmit={handleAddTransaction} />
                <BudgetChart transactions={state.transactions} theme=
{theme} />
            </div>

            {/* L'historique des transactions */}
            <TransactionList
                transactions={state.transactions}
                onDelete={handleDelete}
                onReset={handleReset}
            />
        </div>
    )
```

C'est quoi les props ?

Les props c'est comme des **parametres** qu'on passe a un composant. Par exemple :

- `<TransactionForm onSubmit={handleAddTransaction} />` passe la fonction `handleAddTransaction` au formulaire
- Quand le formulaire appelle `onSubmit(...)`, ca execute `handleAddTransaction` dans App

Analogie : c'est comme donner un talkie-walkie a quelqu'un. Le composant enfant peut "appeler" le parent via ce talkie-walkie (la prop callback).

PARTIE 8 : Les 3 composants

8.1 - `TransactionForm.tsx` - Le formulaire

Ce composant gere ses propres champs (nom, montant, type, categorie) en interne avec `useState`. Quand l'utilisateur clique "Ajouter", il appelle la prop `onSubmit` pour remonter les donnees au parent.

```
function TransactionForm({ onSubmit }: FormProps) {
    // 4 useState = 4 champs du formulaire
    const [name, setName] = useState('')              // champ texte
    const [amount, setAmount] = useState('')           // champ nombre
    const [type, setType] = useState<TransactionType>('depense')    //
bouton depense/revenu
    const [category, setCategory] = useState<Category>('alimentation')
// menu deroulant

    function handleSubmit(e: React.FormEvent) {
        e.preventDefault()  // empeche la page de se recharger
(comportement par defaut d'un form)

        // validation : on verifie que les champs sont remplis
        if (!name.trim() || !amount || parseFloat(amount) <= 0) return

        // on appelle la prop du parent avec les valeurs
        onSubmit(name.trim(),
parseFloat(parseFloat(amount).toFixed(2)), type, category)

        // on vide les champs apres l'ajout
        setName('')
        setAmount('')
    }
```

Pourquoi `e.preventDefault()` ?

Par defaut, quand on soumet un formulaire HTML, le navigateur **recharge la page entiere**. Avec React, on ne veut PAS ca (sinon on perd tout l'etat). `preventDefault()` empeche ce comportement.

Le flux de communication

```
[App] ---onSubmit={handleAddTransaction}---> [TransactionForm]
                                              L'utilisateur
remplit et clique
                                              handleSubmit()
s'execute
                                              Appelle
onSubmit(name, amount, type, category)
[App] <-- handleAddTransaction s'execute
  --> fetch POST vers le serveur
  --> dispatch pour mettre a jour le state
  --> React re-affiche tout
```

8.2 - `BudgetChart.tsx` - Le graphique donut

```
function BudgetChart({ transactions, theme }: ChartProps) {
    const chartRef = useRef<HTMLCanvasElement>(null)      // reference
vers le canvas HTML
    const chartInstance = useRef<Chart | null>(null)      // reference
vers l'objet Chart.js
```

C'est quoi `useRef` et pourquoi ?

`useRef` c'est un **tiroir** : on y stocke une valeur qui persiste entre les rendus SANS declencher de re-render.

- `chartRef` : pointe vers l'element `<canvas>` du HTML (pour que Chart.js sache OU dessiner)
- `chartInstance` : stocke l'objet Chart.js pour pouvoir le **detruire** avant d'en creer un nouveau

Pourquoi detruire le chart avant d'en recreer ?

Si on cree un nouveau chart sans detruire l'ancien, Chart.js affiche les 2 en meme temps, ca superpose les graphiques et ca fait des bugs visuels.

8.3 - `TransactionList.tsx` - L'historique

```
function TransactionList({ transactions, onDelete, onReset }:
ListProps) {
    const [filter, setFilter] = useState<Category | 'tout'>('tout')

    // on filtre les transactions selon la categorie selectionnee
    const filtered = filter === 'tout'
        ? transactions                                    // si "tout",
on affiche tout
        : transactions.filter(t => t.category === filter) // sinon, on
garde que la categorie
```

Le rendu conditionnel (ternaire)

`condition ? valeurSiVrai : valeurSiFaux`

C'est un raccourci pour un if/else. Ici : "si le filtre est 'tout', affiche tout, sinon filtre".

PARTIE 9 : `src/index.css` - Le design

9.1 - Les variables CSS et le theme

```
:root {
    --bg-primary: #f5efe6;        /* couleur de fond principale */
    --text-primary: #3e2c1c;      /* couleur du texte */
    --accent: #8b6f47;            /* couleur d'accentuation (boutons,
titres) */
    --income-color: #2e8b57;      /* vert pour les revenus */
    --expense-color: #c0392b;     /* rouge pour les depenses */
}
```

C'est quoi `:root` et les variables CSS ?

`:root` represente l'element `<html>`. Les variables CSS (qui commencent par `--`) sont des "boites" ou on stocke des valeurs. Ensuite on les utilise partout avec `var()` :

```
.bouton {
    color: var(--accent);   /* ca prend la valeur #8b6f47 */
}
```

Comment marche le theme sombre ?

```
[data-theme="dark"] {
    --bg-primary: #1a1410;    /* fond sombre */
    --text-primary: #e8dfd5;  /* texte clair */
}
```

Quand React met `data-theme="dark"` sur `<html>`, les variables sont **ecrasees**. Tous les elements qui utilisent `var(--bg-primary)` changent automatiquement de couleur. PAS BESOIN de modifier chaque element un par un !

9.2 - Le glassmorphisme (l'effet verre depoli)

```
.panel {
    background: rgba(210, 190, 165, 0.35);   /* fond semi-transparent
*/
    backdrop-filter: blur(12px);               /* flou de ce qui est
derriere */
    border-radius: 14px;                       /* coins arrondis */
}
```

`rgba()` permet de definir une couleur avec de la transparence. Le 4eme chiffre (0.35) c'est l'opacite : 0 = invisible, 1 = opaque.
`backdrop-filter: blur()` floute ce qui est derriere l'element, ca donne un effet "verre".

9.3 - Le responsive (adaptation mobile)

```
.main-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;     /* 2 colonnes egales */
}

@media (max-width: 700px) {
    .main-grid {
        grid-template-columns: 1fr;      /* 1 seule colonne sur mobile
*/
    }
}
```

`@media` c'est une condition CSS : "si l'ecran fait moins de 700px de large, applique ces styles". Ca permet a l'app de bien s'afficher sur telephone.

PARTIE 10 : Resume et Glossaire

Le flux complet d'ajout d'une transaction

```
1. L'utilisateur remplit le formulaire dans TransactionForm
2. Il clique "Ajouter" -> handleSubmit() s'execute
3. e.preventDefault() empeche le reload de la page
4. Validation des champs (nom pas vide, montant > 0)
5. TransactionForm appelle la prop onSubmit(name, amount, type,
category)
6. App recoit l'appel dans handleAddTransaction()
7. On cree l'objet Transaction avec un id unique
8. fetch() envoie un POST au backend Express
9. Vite proxy redirige /api vers localhost:3001
10. Express recoit la requete, ecrit dans transactions.json
11. Express repond 201 (Created)
12. dispatch({ type: 'ADD_TRANSACTION' }) met a jour le state
React
13. React detecte le changement d'etat
14. React re-render TOUS les composants qui dependent de
transactions
15. Le solde, le graphique et l'historique sont mis a jour
```

Glossaire des termes importants

Terme	Definition simple
Composant	Une brique d'interface reutilisable (un bouton, un formulaire...)
Hook	Fonction speciale React qui commence par `use`
State	Les donnees qui changent et declenchent un re-render
Props	Les parametres passes d'un composant parent a un enfant
Dispatch	Envoyer une action au reducer pour modifier le state
Reducer	Fonction qui recoit un etat + une action et retourne le nouvel etat
JSX	Syntaxe qui melange HTML et JavaScript dans React
API REST	Convention d'organisation des URLs d'un serveur
Middleware	Fonction qui s'execute avant chaque requete (ex: cors, json parser)
Proxy	Intermediaire qui redirige les requetes vers un autre serveur
CORS	Securite du navigateur qui bloque les requetes entre domaines differents
DOM	L'arbre HTML de la page que React manipule

Tableau de reference des fichiers

Fichier	Role	Tourne ou ?
`package.json`	Dependances + scripts	--
`index.html`	Squelette HTML	Navigateur
`vite.config.ts`	Config Vite + proxy	Outil de dev
`server.js`	API REST backend	Node.js (terminal)
`data/transactions.json`	Stockage des donnees	Fichier sur le disque
`src/main.tsx`	Point d'entree React	Navigateur
`src/App.tsx`	Composant principal	Navigateur
`src/components/TransactionForm.tsx`	Formulaire d'ajout	Navigateur
`src/components/BudgetChart.tsx`	Graphique donut	Navigateur
`src/components/TransactionList.tsx`	Historique + filtres	Navigateur
`src/index.css`	Tous les styles CSS	Navigateur

Commandes utiles

```
npm install          # installer les librairies (UNE SEULE FOIS)
npm run server       # lancer le backend (port 3001)
npm run dev          # lancer le frontend (port 5173)
npm run build        # compiler pour la production
```